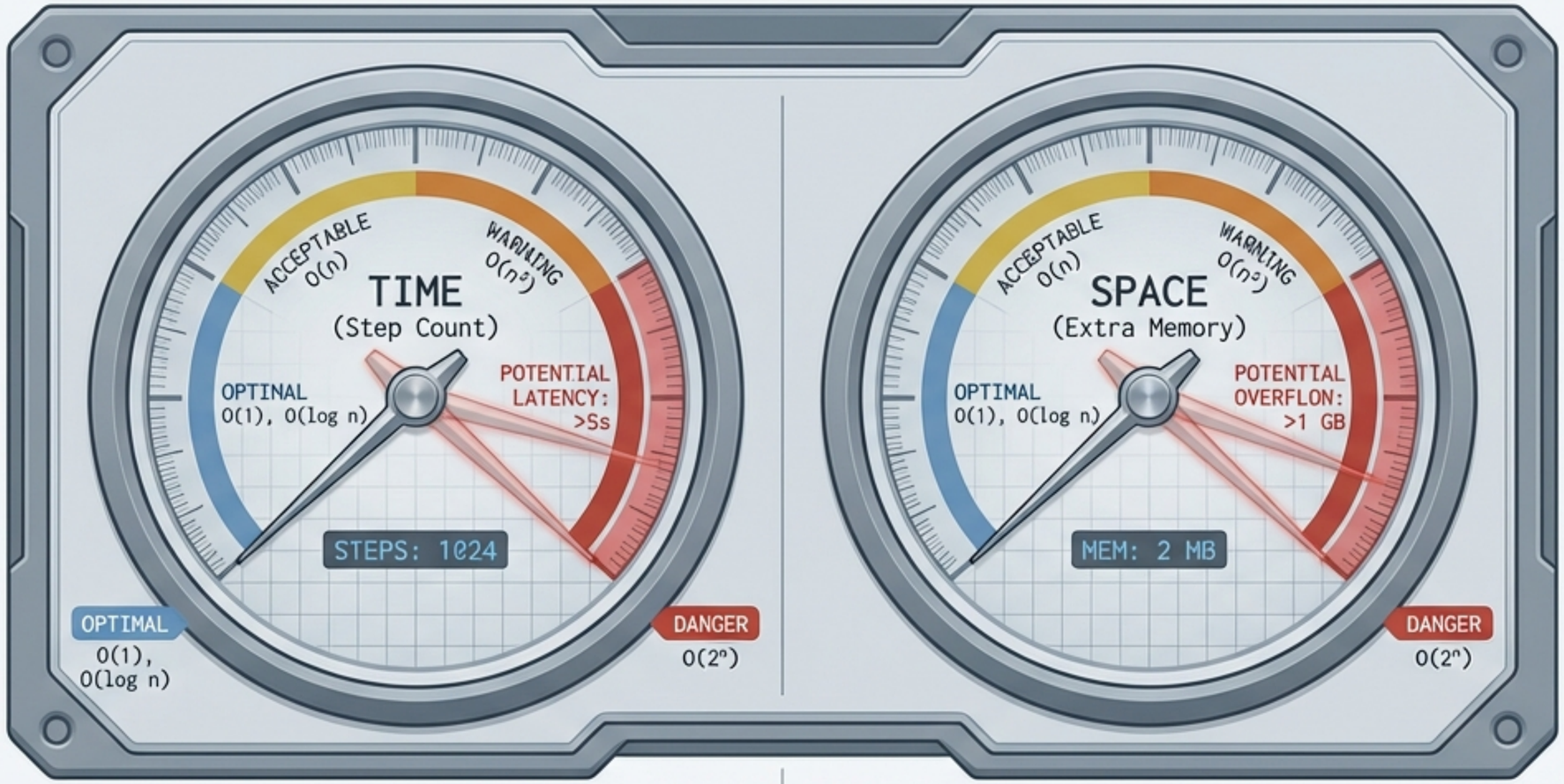
The Cost of Computation

Bridging the gap between code that works and code that scales.




```
> Loading ADP470S Syllabus Profiles... [OK] | Target: Algorithm Efficiency.
```

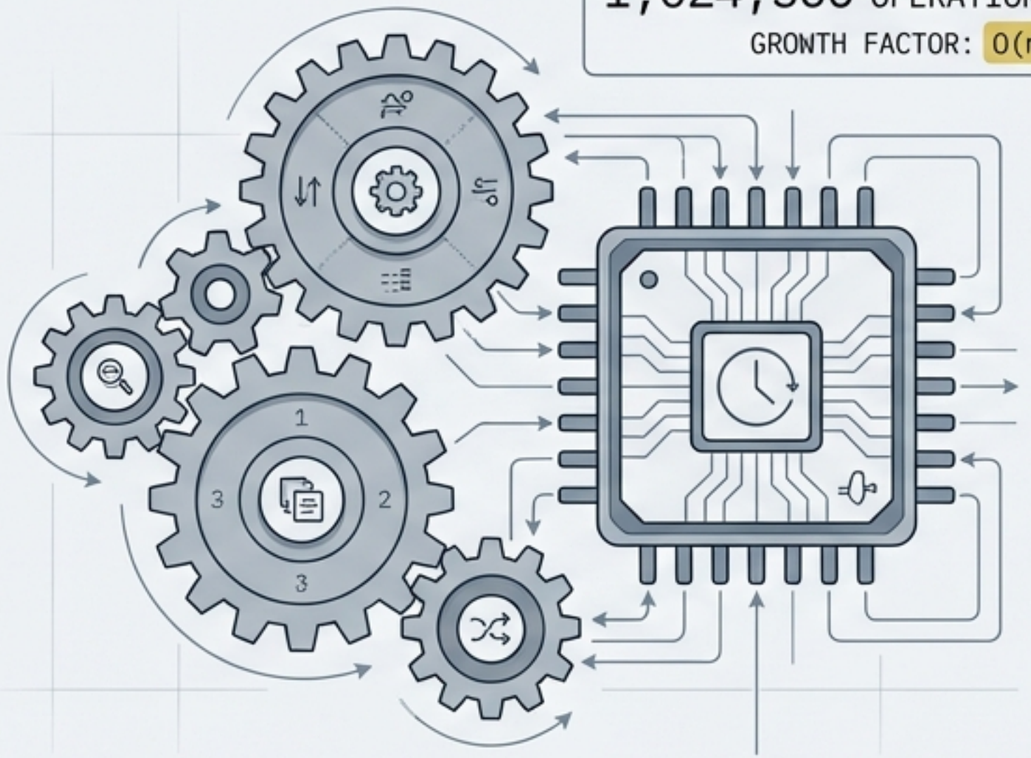
The dual currencies of execution: Time and Space

1. Step Count

TELEMETRY: CYCLE COUNT

1,024,500 OPERATIONS

GROWTH FACTOR: O(n)



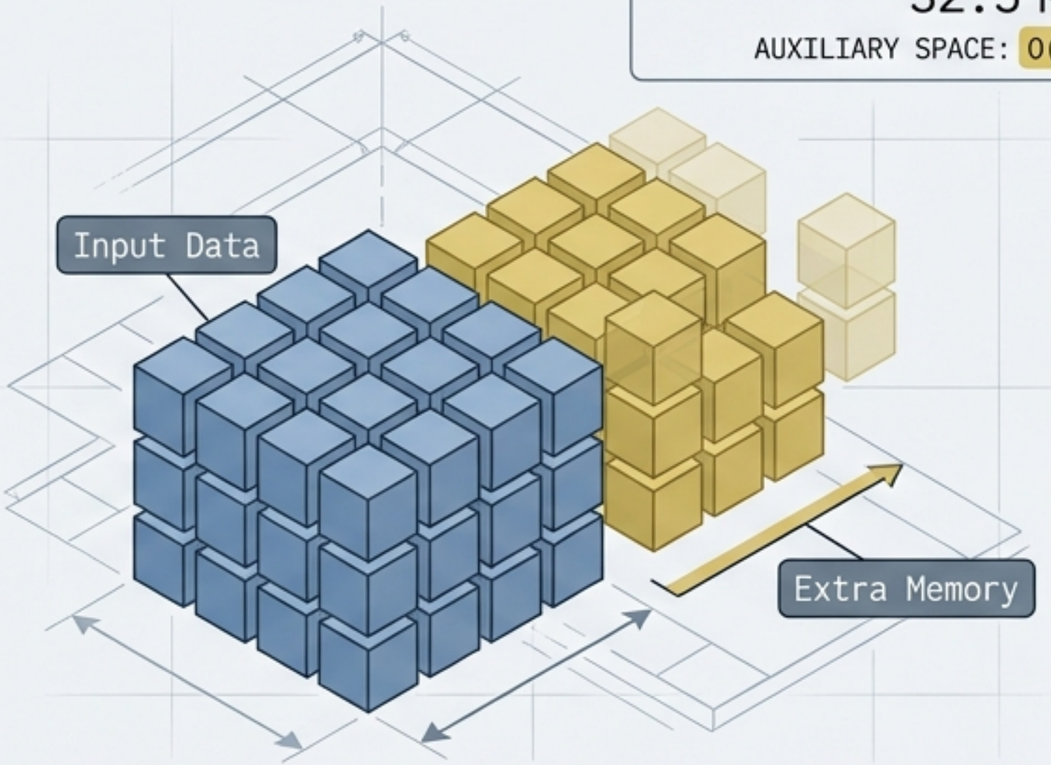
The total number of primitive operations (additions, assignments, comparisons) an algorithm executes. Growth is measured against input size *n*.

2. Extra Memory

TELEMETRY: MEMORY ALLOCATION

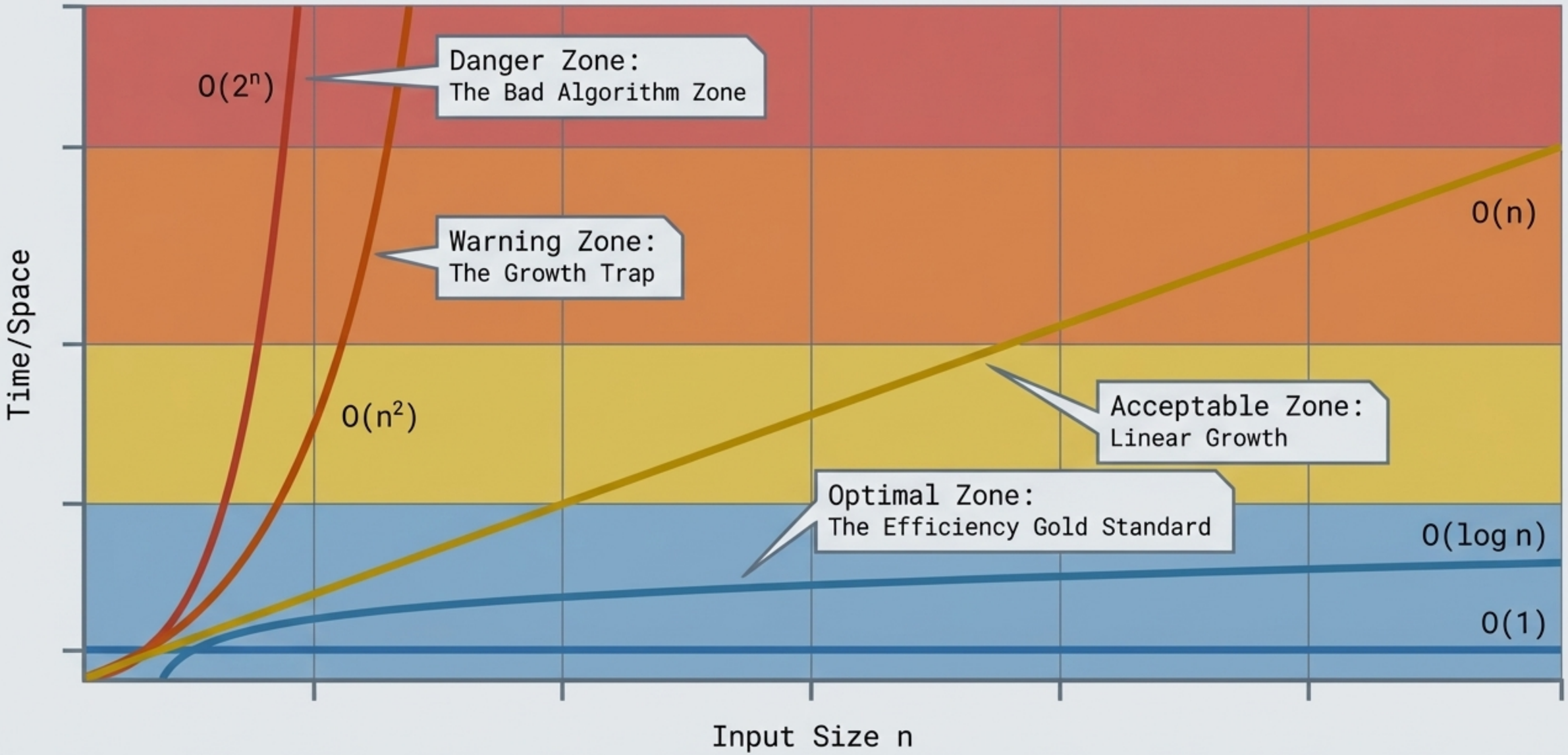
32.5 MB

AUXILIARY SPACE: O(n)

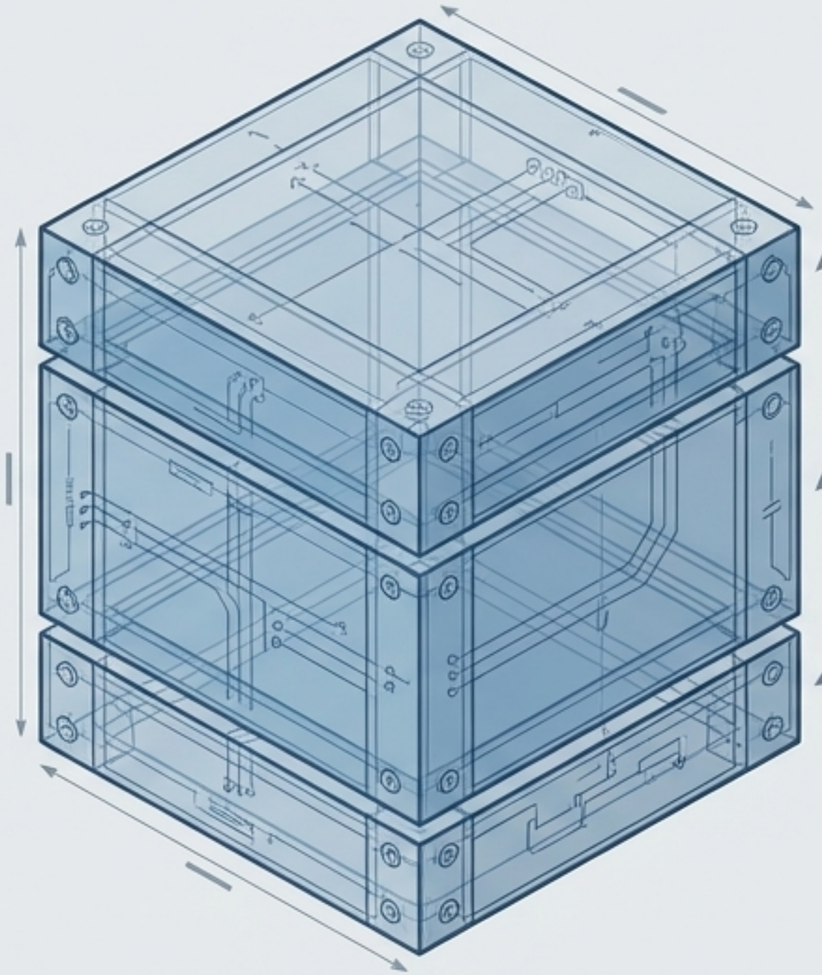


The additional footprint consumed beyond the input data. In recursive contexts, this is strictly dictated by the allocation of stack frames.

Tracking scale through Big-O notation



Anatomy of a single stack frame



Parameters

Values passed into the method.

Local Variables

Variables declared within the method body.

Return Address

The precise code location where control resumes after popping.

TELEMETRY

WARN: Stack Overflow Error.

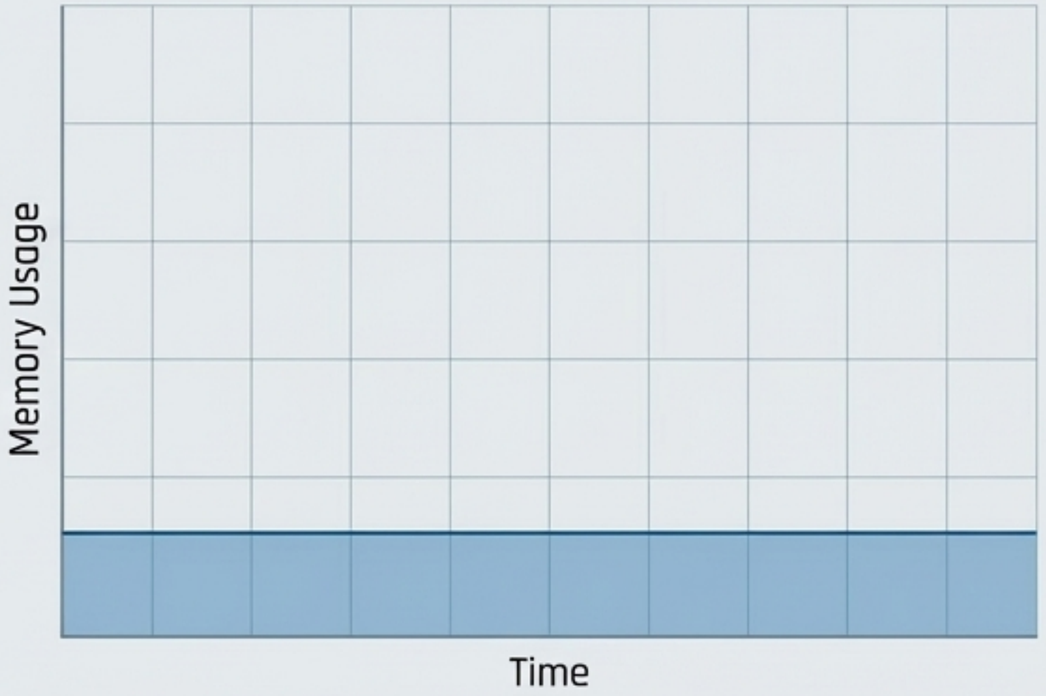
Occurs when memory is exhausted due to missing base cases or excessive recursion depth.

The call stack footprint: Iteration vs. Recursion

Iterative Approach

Space: O(1)

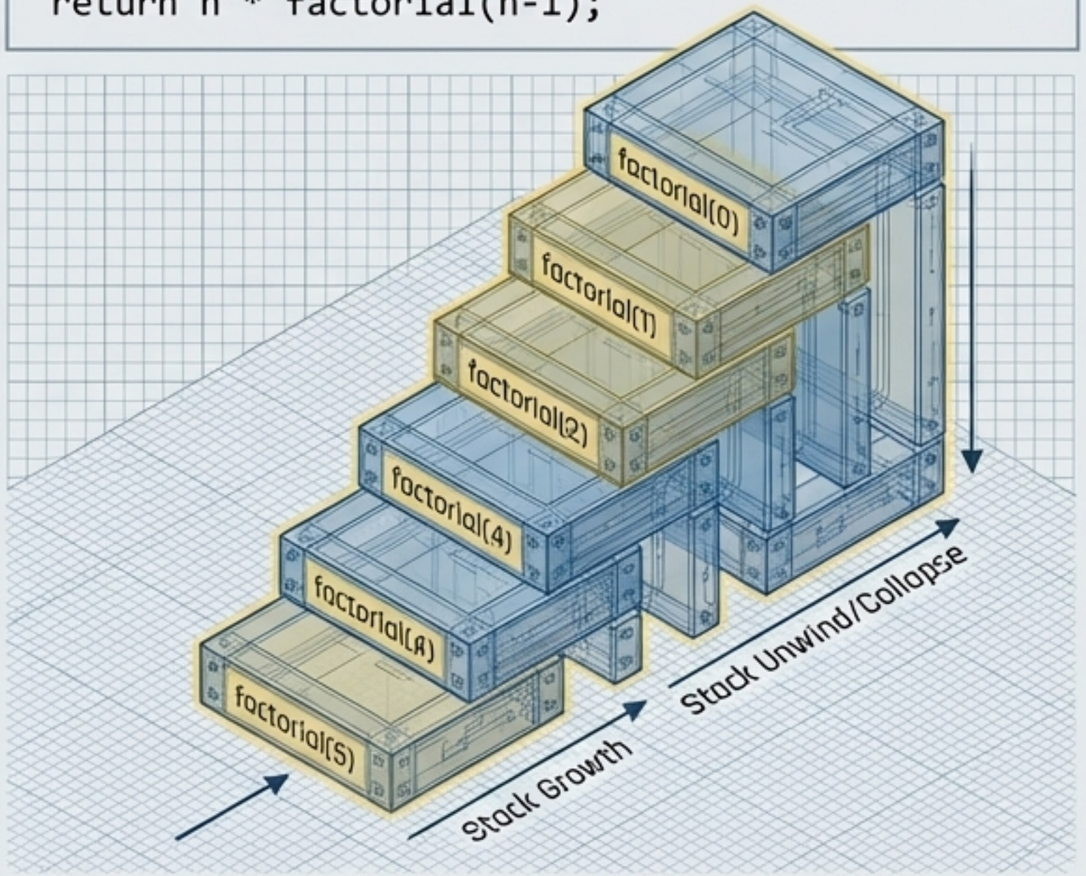
```
int result = 1;
for(int i=1; i<=n; i++) result *= i;
```



Recursive Approach

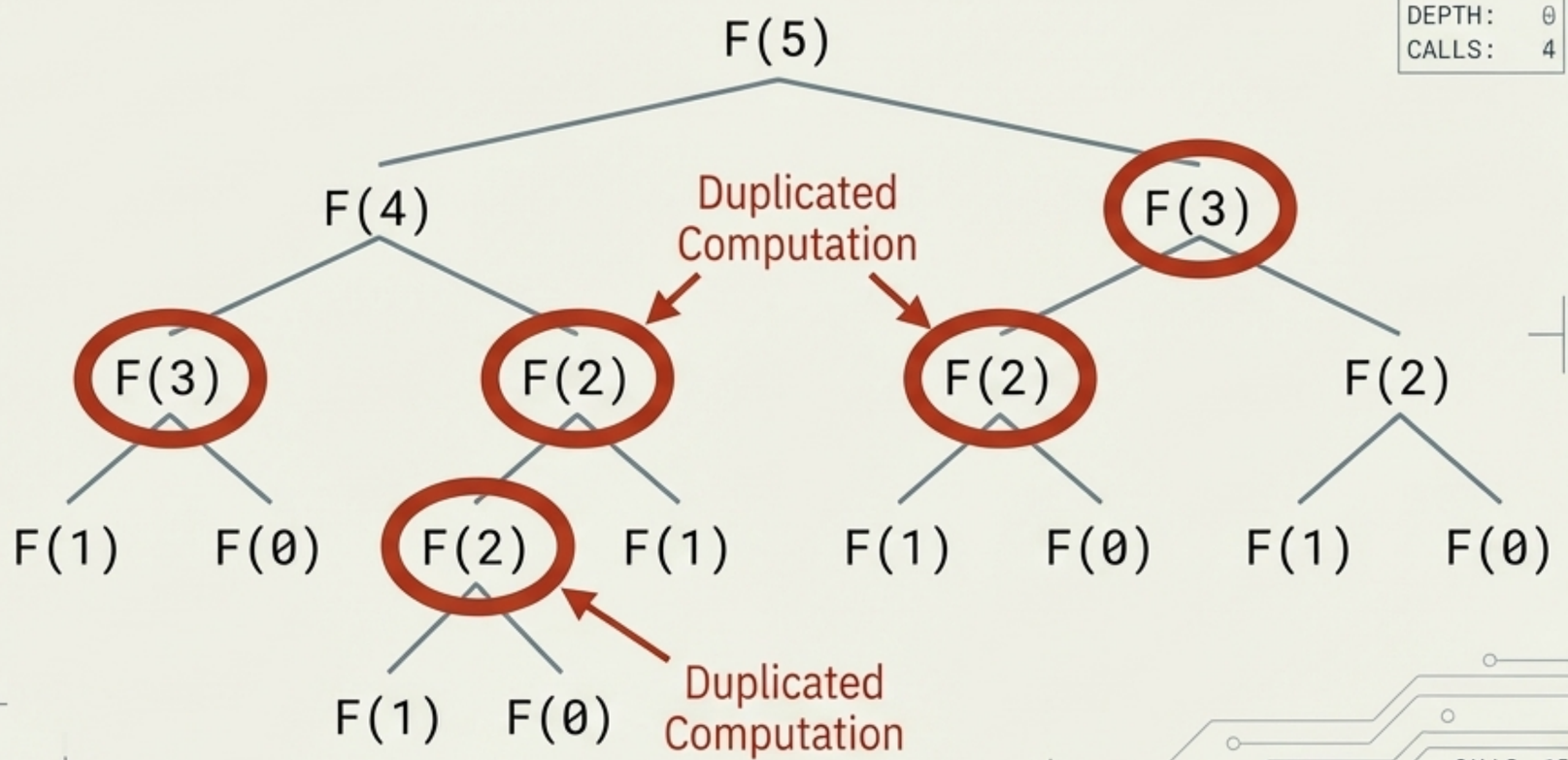
Space: O(n)

```
if (n==0) return 1;
return n * factorial(n-1);
```



The Fibonacci Trap: Exponential redundancy

$$F(n) = F(n-1) + F(n-2)$$



DEPTH: 5
CALLS: 15

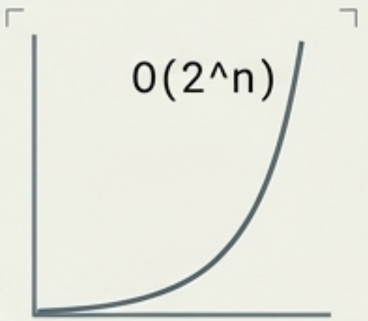
DEPTH: 0
CALLS: 4

CALLS: 15

TELEMETRY WARNING

⚠ CRITICAL: O(2^n) Exponential Time.

Approximately 2^n total function calls to calculate a single integer.



Iteration achieves this in O(n) linear time by storing previous values, avoiding redundant work.

Diagnostic Matrix: Evaluating the execution approach

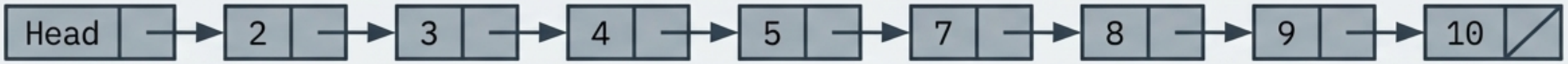
	Recursive Approach	Iterative Approach
Step Count	Grows with depth (constant work per call).	Linear loop `O(n)`.
Extra Memory	`O(n)` Linear frame allocation.	`O(1)` Constant reuse.
Readability	Mirrors mathematical definitions; concise.	Verbose, but avoids call-overhead.

The Verdict: While recursion offers mathematical elegance, iterative solutions are structurally superior in production environments due to O(1) space and zero frame-allocation overhead.

Defining recursive data structures

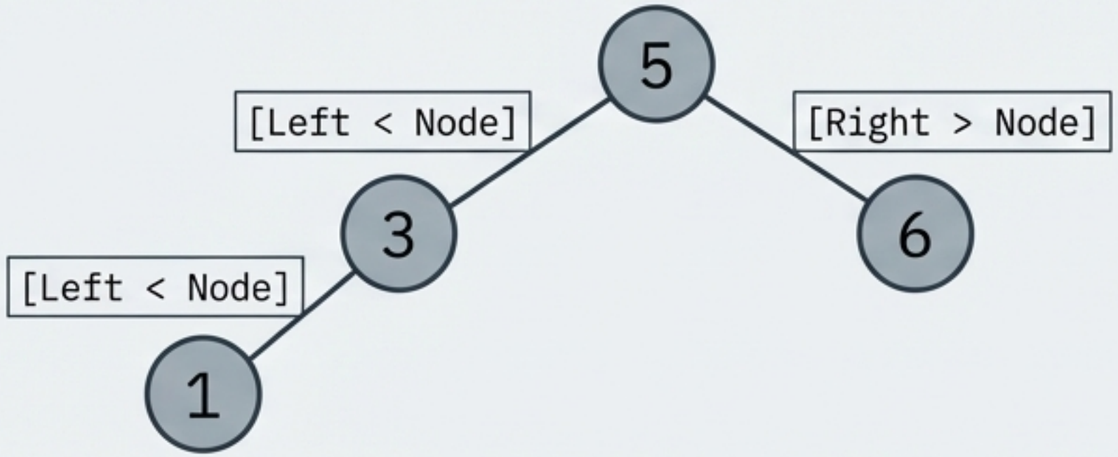
Linear Linked Lists

A list is recursively defined as a Head element followed by a sub-list (Tail).



1 → 2 → 3 → 4 → 4 → 5 → 7 → 8 → 9 → 10

Anatomy of a BST (Binary Search Tree)



CONSTRAINT

Duplicates are strictly prohibited. Max two children per node.

Finding data: The search efficiency divide

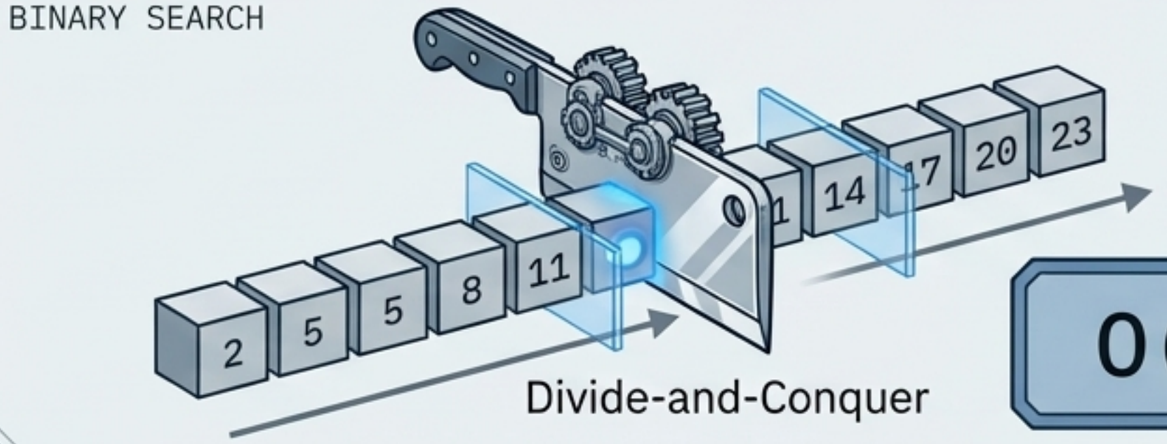
Linear Search

Iterates one element at a time until the target is found.

Best Case: O(1)

Worst/Average Case: O(n)

Binary Search



O(log n)

PREREQUISITE: Data must be sorted/indexed.

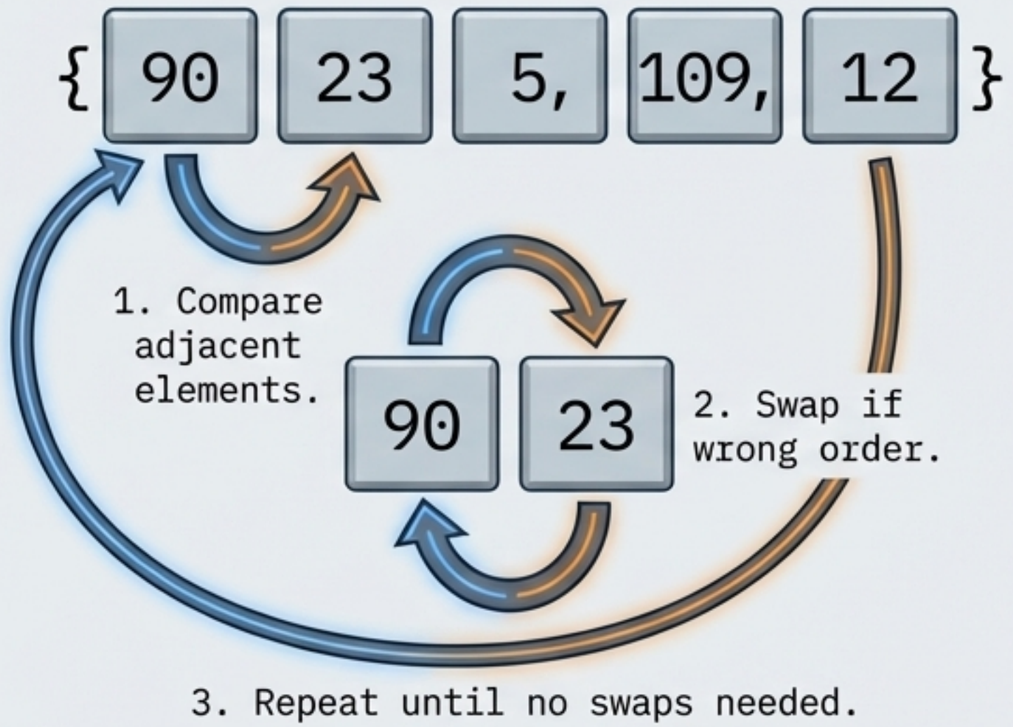
Trees (BST) are natively searchable in log time.

The mechanics of Bubble Sort

The Code

```
for(int i=0; i < n-1; i++) {
    for(int j=0; j < n-i-1; j++) {
        // Compare and swap
    }
}
```

The Mechanism



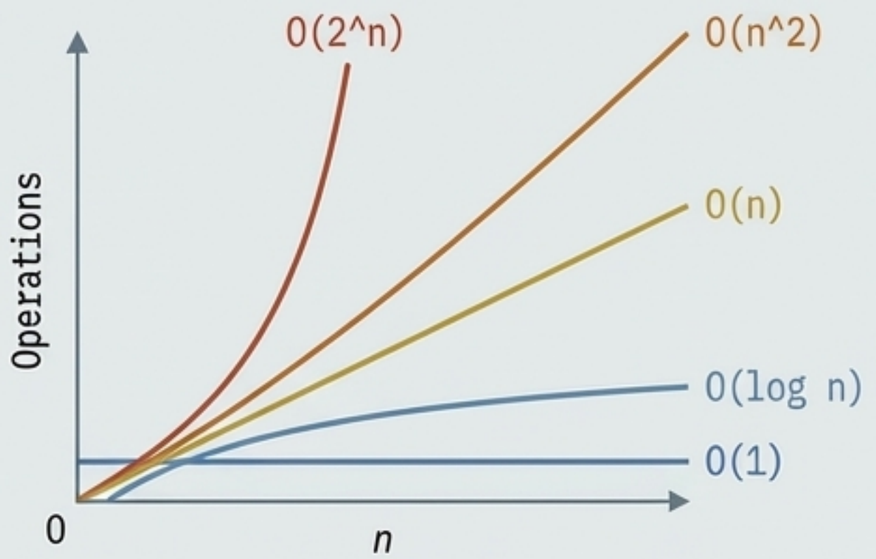
WARNING: Nested loops lock this algorithm into an O(n^2) average and worst-case time complexity. Simple, but structurally slow for scale.

The Sorting Algorithm Showdown

	Type	Best Case	Worst Case	Notes
Bubble Sort	Iterative	O(n) if modified	O(n^2)	Swaps adjacent elements.
Selection Sort	Iterative	O(n^2)		Selects smallest, moves to front. Inefficient for scale.
Insertion Sort	Iterative	O(n)	O(n^2)	Good for small/nearly sorted lists.
Merge Sort	Recursive	O(n log n)	O(n log n)	Fast divide & conquer. Uses extra memory.
Quick Sort	Recursive	O(n log n)	O(n^2)	Usually fastest in practice, but unstable worst case.

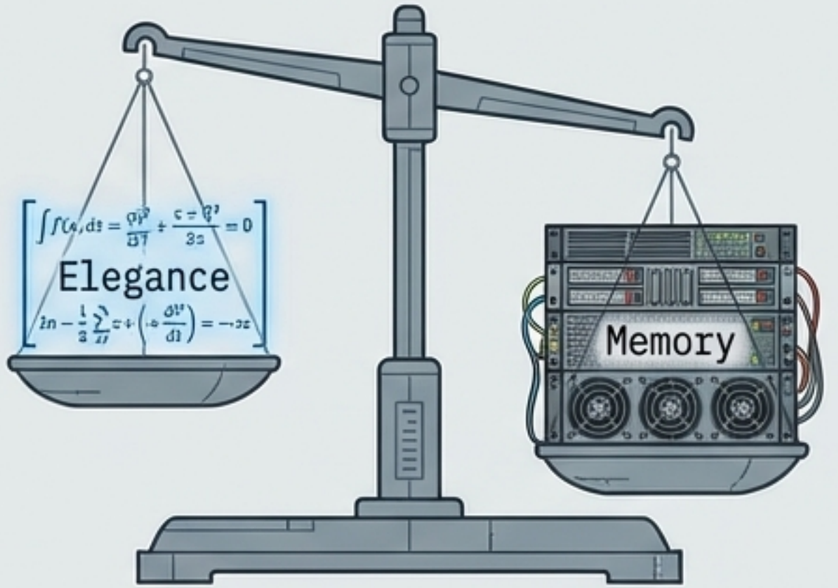
The Developer's Efficiency Cheatsheet

The Scale Reality



Always design for `n`. `O(1)` and `O(log n)` are the targets. Nested loops trigger `O(n^2)` degradation.

The Recursion Rule



Just because you can use recursion, doesn't mean you should. Beware the hidden `**stack frame memory footprint**` and exponential redundancy.

Evaluate code not by subjective elegance, but by measurable telemetry.

Code that works is the baseline. Code that scales is the objective.